



CL1002 <i>Programming Fundamentals Lab</i>	Lab 03 Introduction to C programming language
---	---

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Fall 2024

AIMS AND OBJECTIVES

Aims:

- Equip students with the foundational knowledge needed to write and understand C programs.
- Enable students to develop efficient and organized code using C language constructs.
- Foster the ability to apply C programming skills to solve real-world problems.
- Prepare students for more advanced topics in programming by solidifying their understanding of basic C concepts.

Objectives:

- Introduce the basic syntax and structure of the C programming language.
- Teach students how to write and compile simple C programs.
- Familiarize students with fundamental C programming concepts, such as variables, data types, operators, and control structures.
- Develop problem-solving skills by writing C programs to solve basic computational problems.
- Understand the use of functions in C to modularize and organize code.

INTRODUCTION

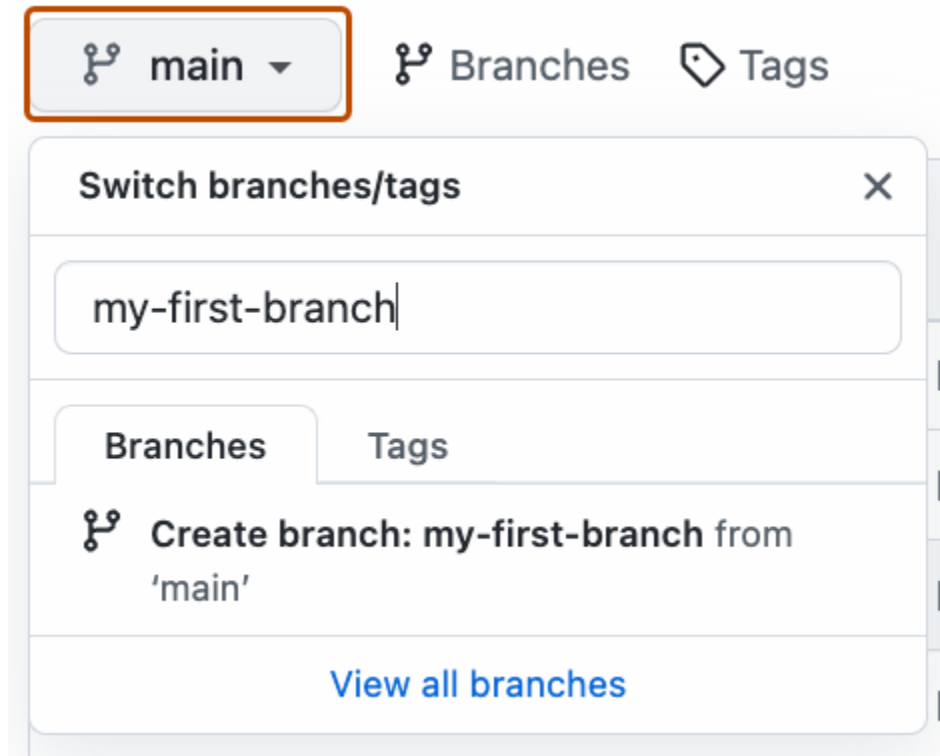
In this lab we will introduce the following concepts.

- Creating a Branch in a Repository
- Creating a Pull Request of the created branch.
- Merging branches to main repository.
- Introduction to Integrated Development Environment (IDE).
- Introduction to C-Programming Language (Basic Structure, Inputs and Outputs, Variables, Data types, Format specifiers, escape sequences, Precision)

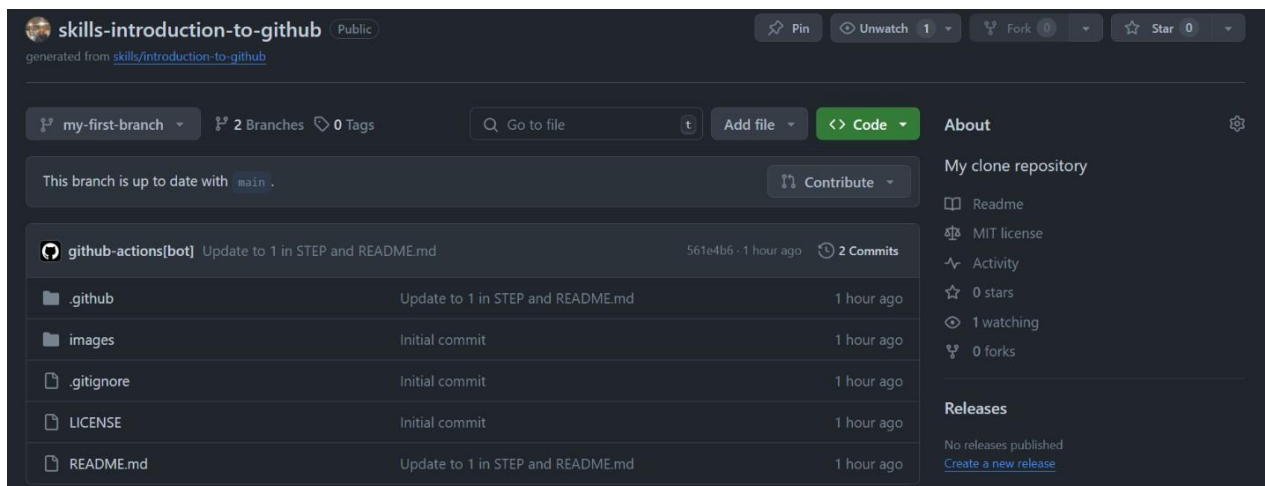
CREATE A BRANCH IN THE REPOSITORY

What is a branch? A branch is a parallel version of your repository. By default, your repository has one branch named **main** and it is the definitive branch. Creating additional branches allows you to copy the **main** branch of your repository and safely make any changes without disrupting the **main** project. Many people use branches to work on specific features without affecting any other parts of the project.

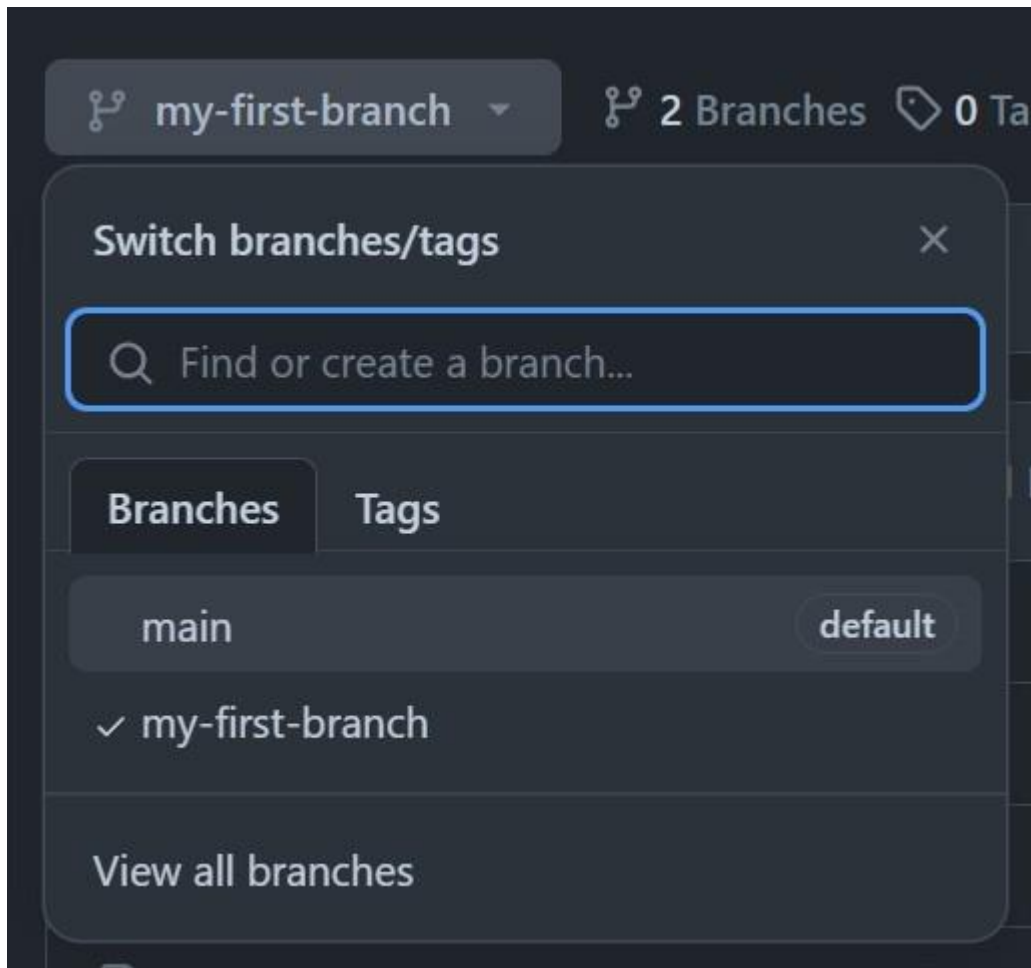
Branches allow you to separate your work from the **main** branch. In other words, everyone's work is safe while you contribute.



While in your repository click on the main tab and enter your branch name and hit the enter key or select the **create branch** option. {Type a suitable name}. You should see a screen something like this.



When creating a branch, it duplicates all the content that is in the main branch to the branch you have created. You can create as many branches as you like. To go back to your main branch just use the drop down menu.



Note that anything that is updated in the main branch will prompt a message in the other branches that it is x commits away (The x represents the number of commits in that particular branch) and anything that is updated in the other branch needs to be merged with the main branch to update the main repository.

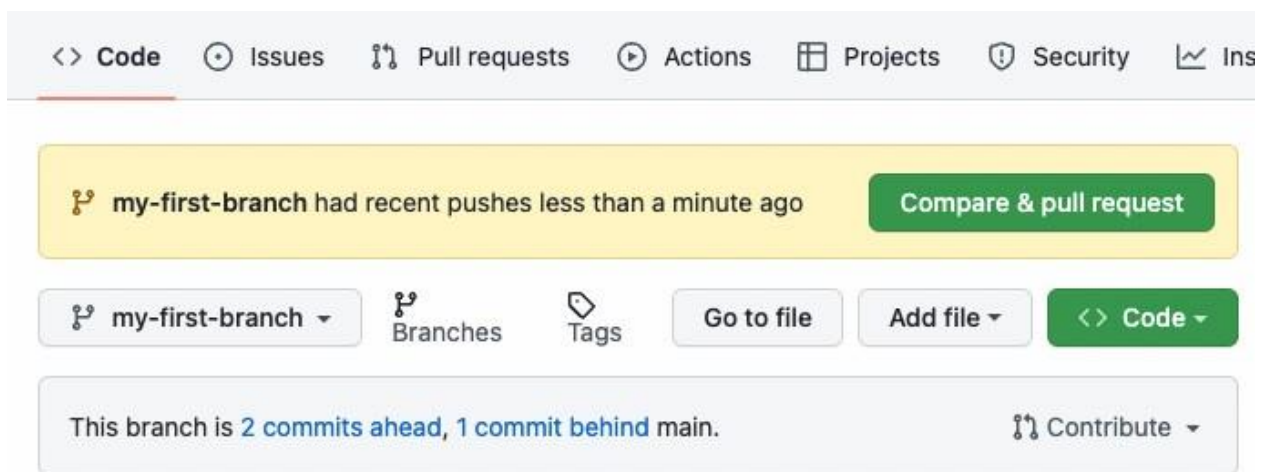
OPENING A PULL REQUEST

Nice work making that pull request! ✨

Now that you have made a change to the project and created a commit, it's time to share your proposed change through a pull request!

What is a pull request?: Collaboration happens on a [pull request](#). The pull request shows the changes in your branch to other people and allows people to accept, reject, or suggest additional changes to your branch. In a side-by-side comparison, this pull request is going to keep the changes you just made on your branch and propose applying them to the main project branch.

You may have noticed after your commit that a message displayed indicating your recent push to your branch and providing a button that says **Compare & pull request**.



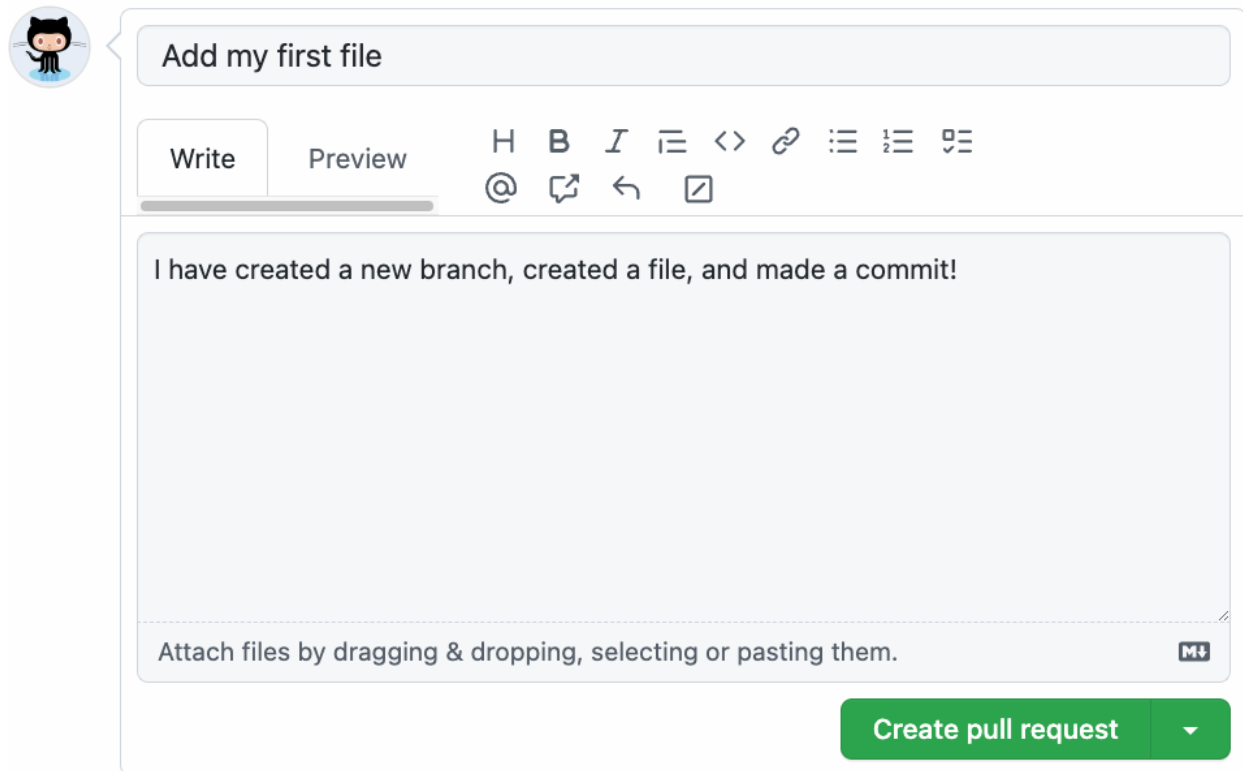
To create a pull request automatically, click **Compare & pull request**, and then skip to step 6 below. If you don't click the button, the instructions below walk you through manually setting up the pull request.

1. Click on the **Pull requests** tab in the header menu of your repository.
2. Click **New pull request**.
3. In the **base:** dropdown, make sure **main** is selected.
4. Select the **compare:** dropdown, and click my-first-branch.



5. Click Create pull request.
6. Enter a title for your pull request. By default, the title will automatically be the name of your branch. For this exercise, let's edit the field to say Add my first file.

7. The next field helps you provide a description of the changes you made. Here, you can add a description of what you've accomplished so far. As a reminder, you have: created a new branch, created a file, and made a commit.



Add my first file

Write

Preview

I have created a new branch, created a file, and made a commit!

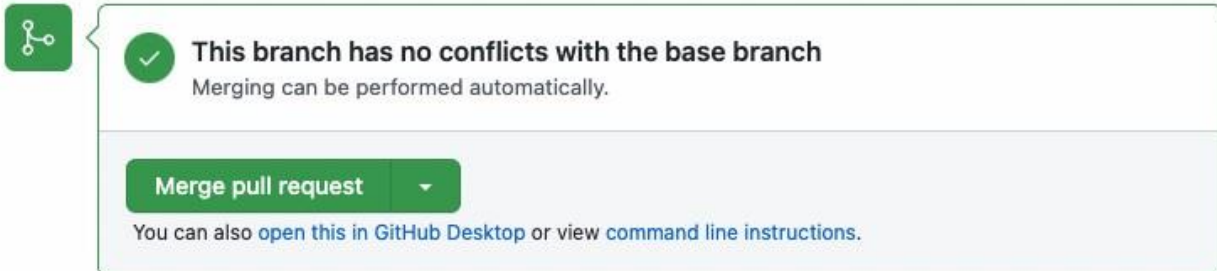
Attach files by dragging & dropping, selecting or pasting them.

Create pull request

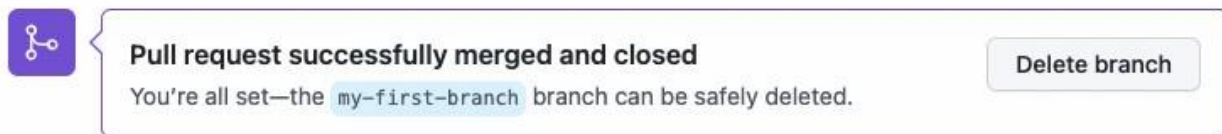
8. Click Create pull request. You will automatically be navigated to your new pull request.

MERGE A PULL REQUEST

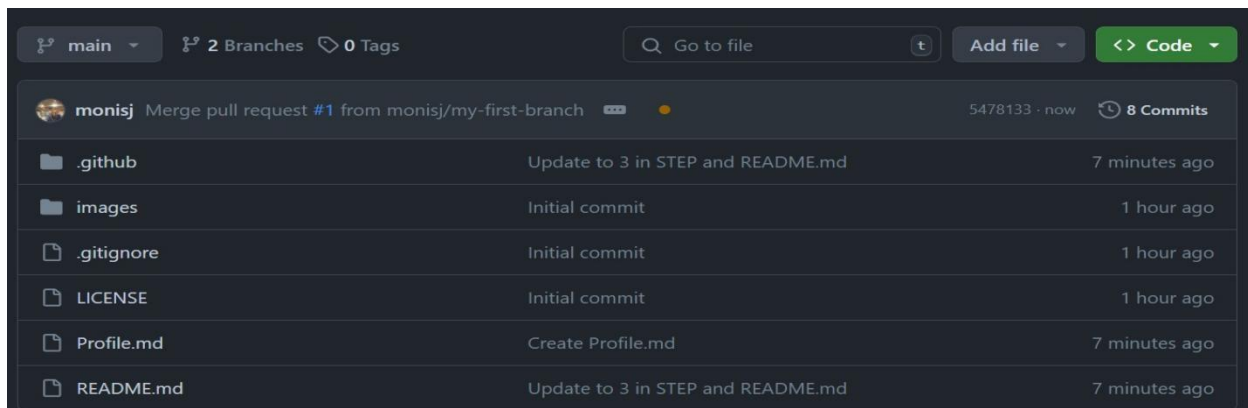
What is a merge?: A [merge](#) adds the changes in your pull request and branch into the main branch. For more information about merges, see "[Merging a pull request](#)."



You would have seen a message like this. If no conflicts occur press the Merge pull request button. Click confirm. Once your branch has been merged, you don't need it anymore. To delete this branch, click **Delete branch**.



Note: deleting a branch helps to simplify your repository but keeping it is no harm as you can still reuse it for testing your work without modifying the main branch.



You should see that all the contents from the other branch are merged into the main branch.

INTRODUCTION TO INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)

- IDE stands for Integrated Development Environment. It is a software application that provides facilities for developing software.
- It consists of tools such as source code editor, automation tools, and debugger.
- For C language programming we will be using Dev-C++.
- Dev-C++ is a full-featured Integrated Development Environment (IDE) for the C/C++ programming language.
- Dev-C++ is free software and is distributed under the GNU General Public License. Thus we can distribute or modify the IDE freely.
- “Bloodshed Software” originally developed it, and Orwell has forked it after it was abandoned by Bloodshed in 2006.
- As similar IDEs, it offers to the programmer a simple and unified tool to edit, compile, link, and debug programs.

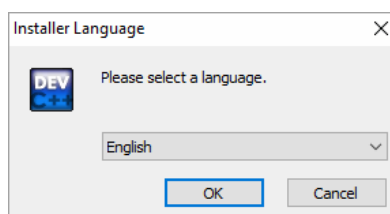
INSTALLATION AND CONFIGURATION OF IDE

We can get the appropriate installable for dev-C++ IDE from <https://www.bloodshed.net/>

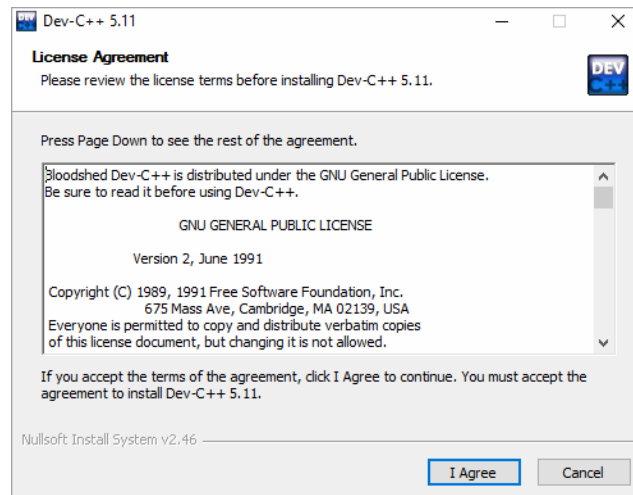
- Let's see the entire installation process now. We have used the installable that comes along with the [C++ compiler](#).

The stepwise installation for Dev-C++ is given below.

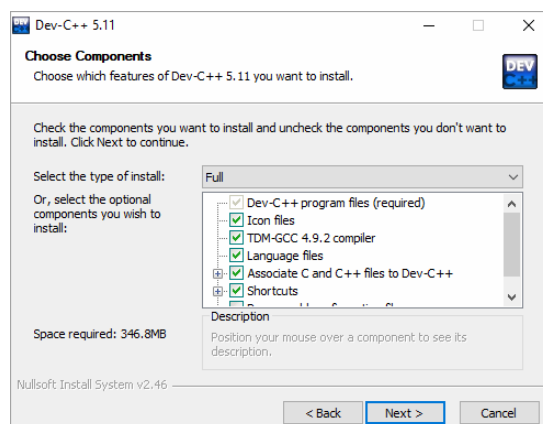
- The first step while we start the installer is to select the language of our choice as shown in the below screenshot.



- Once you select the appropriate language, you have to agree to the license agreement that pops up next.

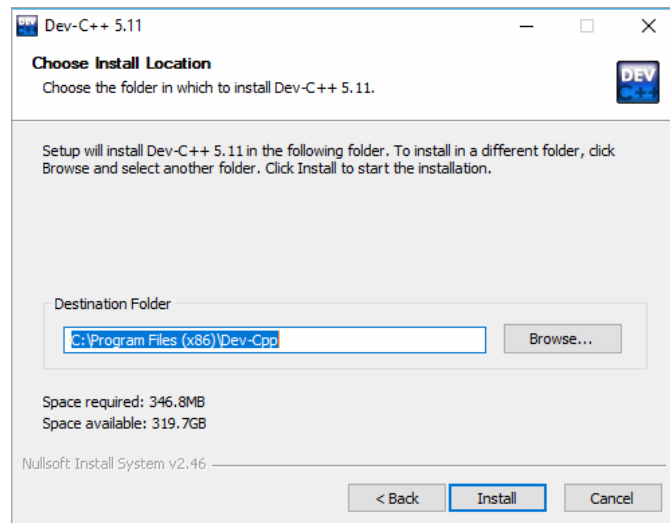


- Next, we are asked to select the components that we need to install as a part of the dev-C++ installation.



As shown in the above screenshot, we are provided with a list of components available for installation and a checkbox against each component. We can check/uncheck each box to indicate which components to install. Click next once the components are selected.

- Now the installer prompts the user for the destination folder where the dev-C++ files/libraries etc. are to be copied.

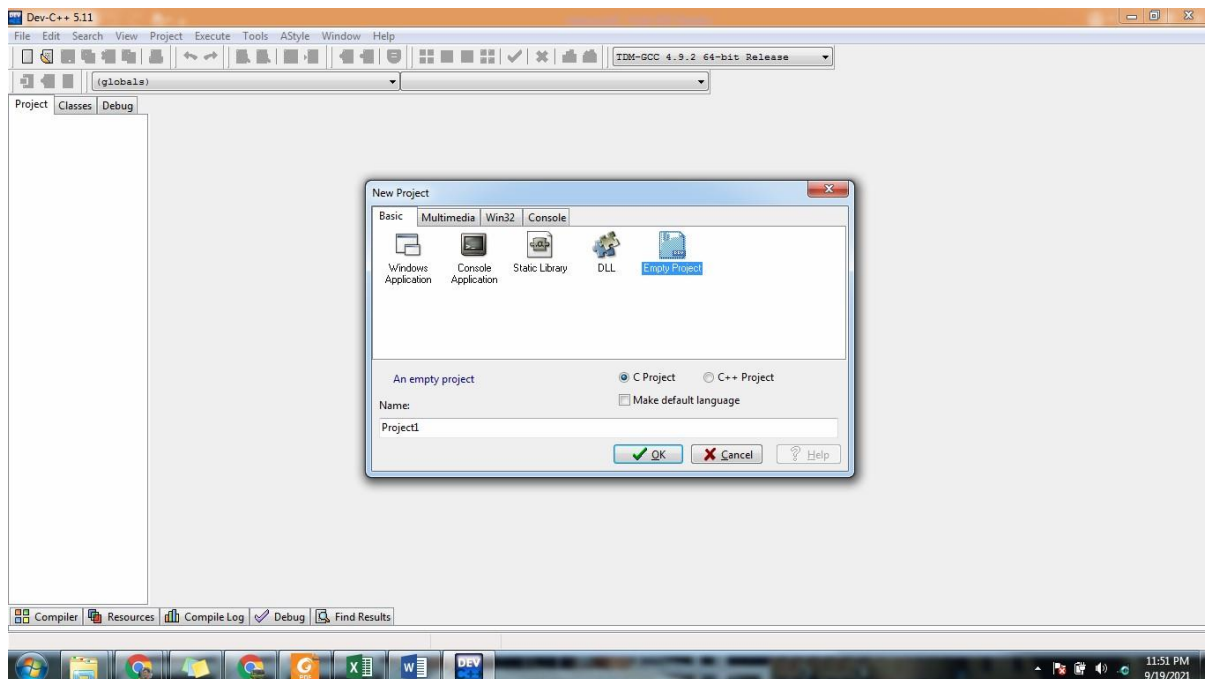


Once we provide the destination folder path, click on Install.

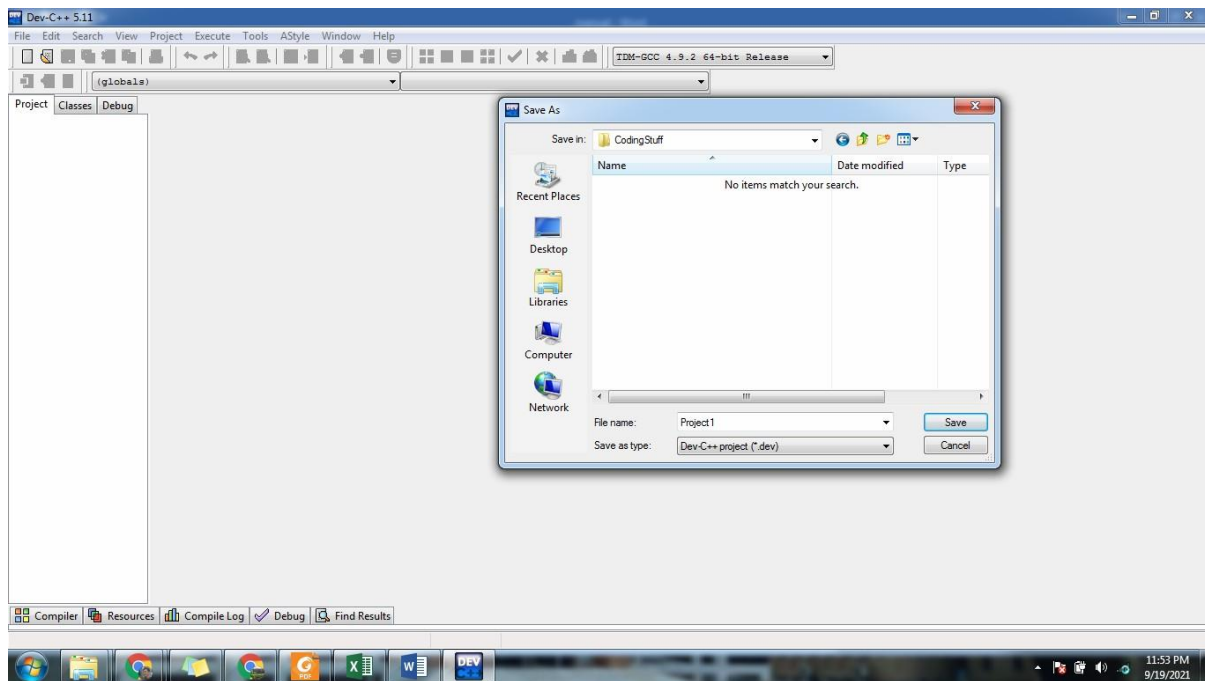
DEVELOPMENT USING THE DEV-C++ IDE.

CREATE A NEW PROJECT

- To create a new project or source file in dev-C++ we need to follow the below steps:
- Click File -> New->Project
- Here, we can specify the project name. Make sure to select the “Empty Project” and also to check the “C++ Project” button.

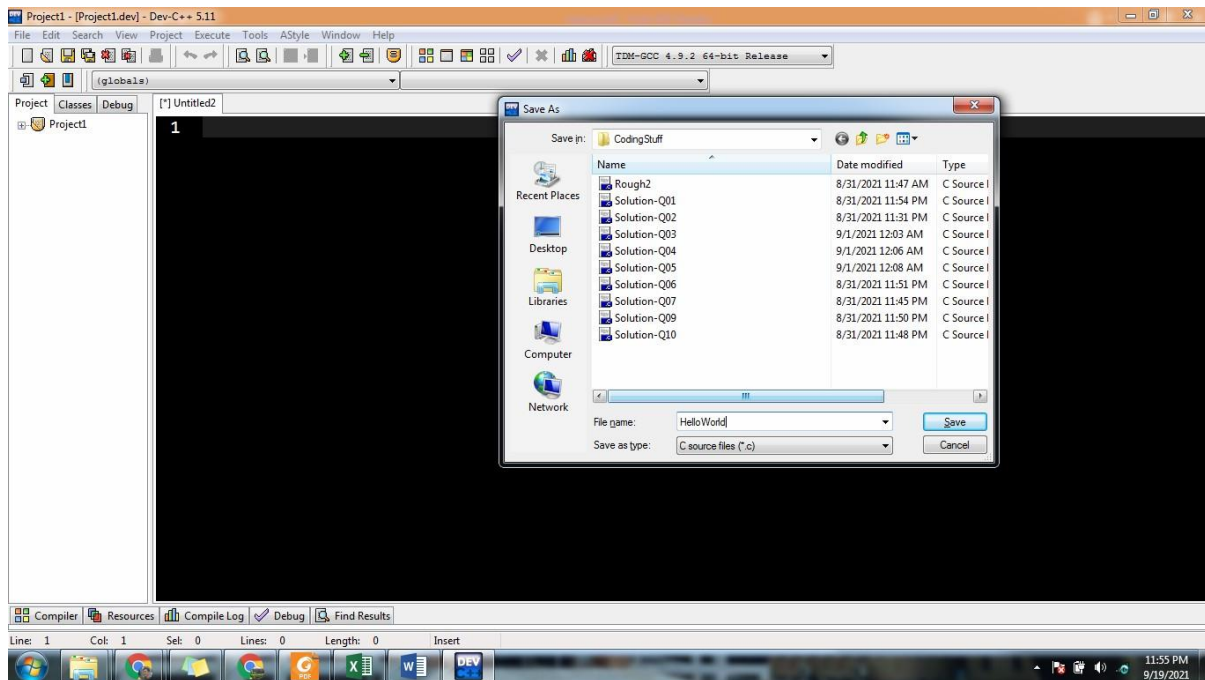


- Once the entire information is provided, we can click ok and the IDE will ask for the path where the project is to be saved. When this is done, a workspace will open with the project explorer on the left-hand side that shows the project we just created.



ADD A SOURCE FILE TO A PROJECT

- Add a new file by clicking **Project ->New File** or Right-click on **Project Name** in the project explorer and click **New File**
- Save the file using CTRL+S or save option in file menu give it a name and choose extension c



HELLO WORLD PROGRAM

- This is the traditional “Hello World” program in C language

```
1  #include<stdio.h>
2
3  main(){
4      printf("Hello World");
5  }
```

Understanding HELLO WORLD Program

- This program uses (that is, it ‘includes’) code from the C-language ‘standard input/output library, stdio, using this statement:

#include <stdio.h>

- The code that starts with the name main is the ‘main function’ – in other words, it is the first bit of code that runs when the program runs. The function name is followed by a pair of parentheses. The code to be run is enclosed between a pair of curly brackets:

```
main() {
}

```

- In this case, the code calls the C printf function to print the string (the piece of text) between double-quotes.

printf("hello world");

COMPILE/BUILD

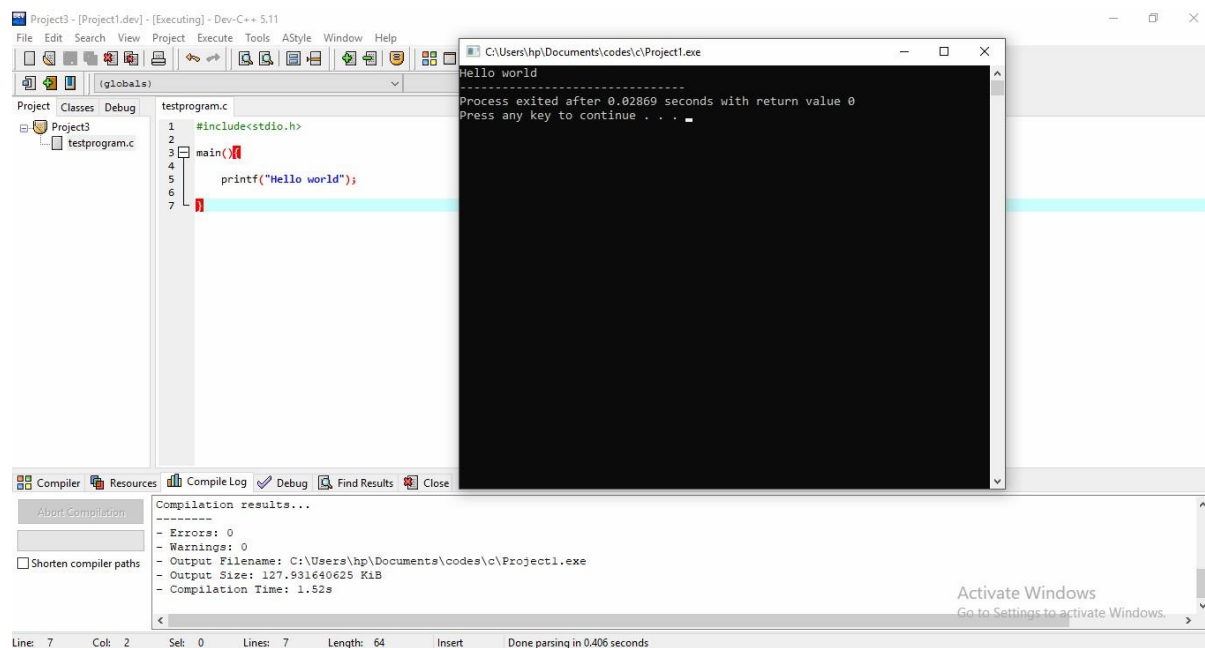
When we have all the code ready for the project, we will now compile and build the project.

Follow the below steps to build and execute the dev C++ project:

- To compile the project, click **Execute -> Compile** (or click F9).
- We can see the compilation status in the “**Compile Log**” tab in the workspace.
- If there are any errors whether syntax or linker errors, then they will appear in the compiler tab.
- Once the project is compiled successfully, we need to run it.

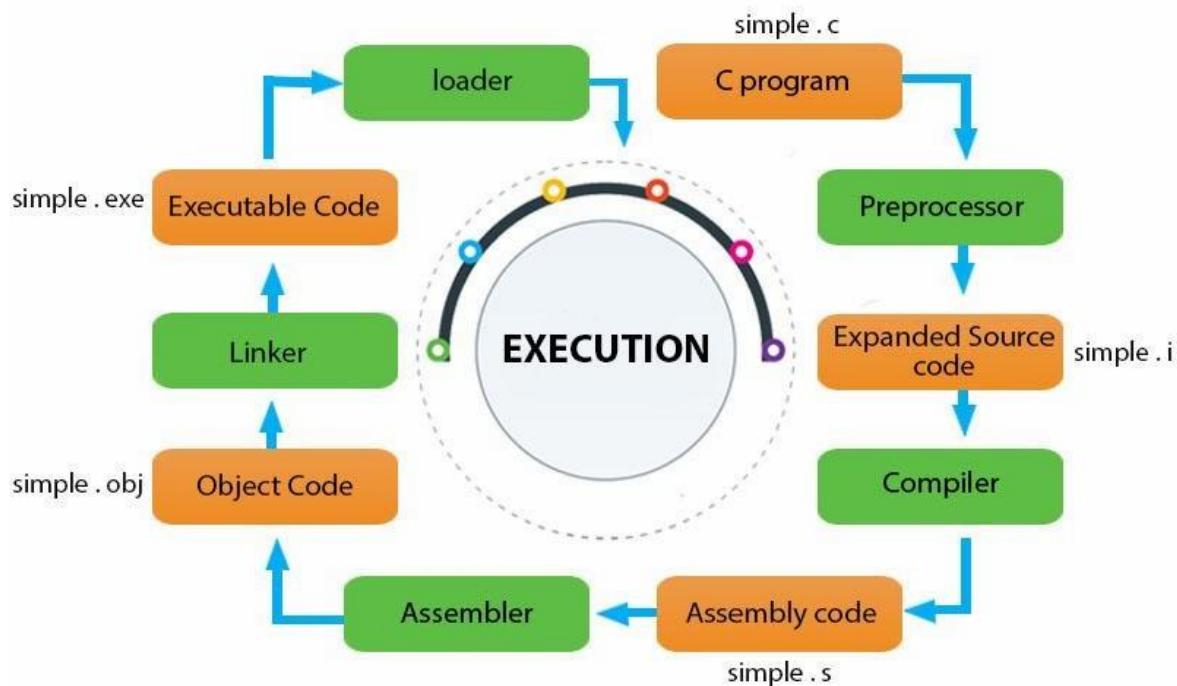
EXECUTE PROJECT

- Click on **Execute ->Run.**(or click F10)
- The console window that gives us the output will be shown in the below screenshot.



FLOW OF EXECUTION OF C PROGRAM

- C is a high-level programming language developed in 1972 by Dennis Ritchie at AT&T Bell Laboratories.
- Programs in C typically go through six phases to be executed. These are: edit, preprocess, compile, link, load and execute.



1. C program (source code) is sent to **preprocessor** first. The preprocessor is responsible to convert preprocessor directives into their respective values. The preprocessor generates an expanded source code.
2. Expanded source code is sent to **compiler** which compiles the code and converts it into assembly code.

Computer programs are written using high-level programming languages. The source code is converted into machine-understandable machine code. A compiler is used for this conversion. Compiler is a translator that converts the source code from high-level programming language to a lower level machine language in order to create an executable program.

3. The assembly code is sent to **assembler** which assembles the code and converts it into object code. Now a simple.obj file is generated.
4. The object code is sent to **linker** which links it to the library such as header files. Then it is converted into executable code.

Library functions are not a part of any C program but of the C software. Thus, the compiler doesn't know the operation of any function, whether it is `printf` or `scanf`. The definitions of these functions are stored in their respective library which the compiler should be able to link. This is what the Linker does. So, when we write `#include`, it includes `stdio.h` library which gives access to Standard Input and Output. The linker links the object files to the library functions and the program becomes a `.exe` file. A simple `.exe` file is generated which is in an executable format.

5. The executable code is sent to **loader** which loads it into memory and then it is executed. After execution, output is sent to console.

Whenever we give the command to execute a particular program, the loader comes into work. The loader will load the `.exe` file in RAM and inform the CPU with the starting point of the address where this program is loaded.

COMPILING A C PROGRAM WITHOUT IDE

We usually use a compiler with a graphical user interface, to compile our C program. This can also be done by using cmd.

STEP 1: Run the command '`gcc -v`' to check if you have a compiler installed. If not you need to download a gcc compiler and install it. You can search for cmd in your windows system to open the command prompt.



```
Command Prompt
Microsoft Windows [Version 10.0.17134.829]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\User> gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=C:/TDM-GCC-64/bin/./libexec/gcc/x86_64-w64-mingw32/5.1.0/lto-wrapper.exe
Target: x86_64-w64-mingw32
Configured with: ../../src/gcc-5.1.0/configure --build=x86_64-w64-mingw32 --enable-targets=all --enable-languages=ada,c,c++,fortran,lto,objc,obj-c++ --enable-libgomp --enable-lto --enable-graphite --enable-cxx-flags=DWINPTHREAD_STATIC --disable-build-with-cxx --disable-build-poststage1-with-cxx --enable-libstdcxx-debug --enable-threads=posix --enable-version-specific-runtime-libs --enable-fully-dynamic-string --enable-libstdcxx-threads --enable-libstdcxx-time --with-gnu-ld --disable-werror --disable-nls --disable-win32-registry --prefix=/mingw64tdm --with-local-prefix=/mingw64tdm --with-pkgversion=tdm64-1 --with-bugurl=http://tdm-gcc.tdragon.net/bugs
Thread model: posix
gcc version 5.1.0 (tdm64-1)

C:\Users\User>
```

Step 2: Create a c program and store it in your system.

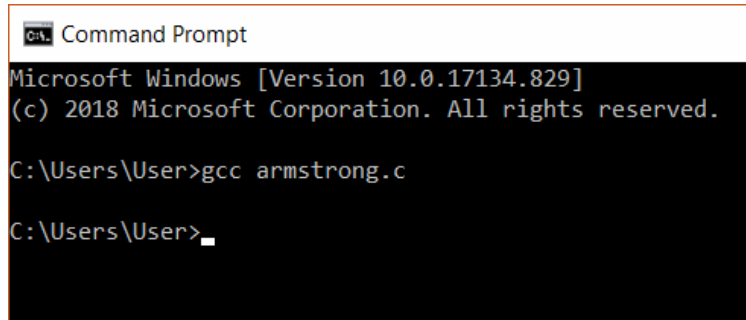
STEP 3: Change the working directory to where you have your C program. You can do that by using the command '`cd`', which changes the directory. We need to pass the name of the directory in which the program is stored.

Example: `cd Desktop`

Our program is already in the user directory so we don't have to change it.

STEP 4:

The next step is to compile the program. To do this we need to use the command `gcc` followed by the name of the program we are going to execute. In our case, we will use `Armstrong.c`. Note that `Armstrong.c` is the name of the file.



```
C:\> Command Prompt
Microsoft Windows [Version 10.0.17134.829]
(c) 2018 Microsoft Corporation. All rights reserved.

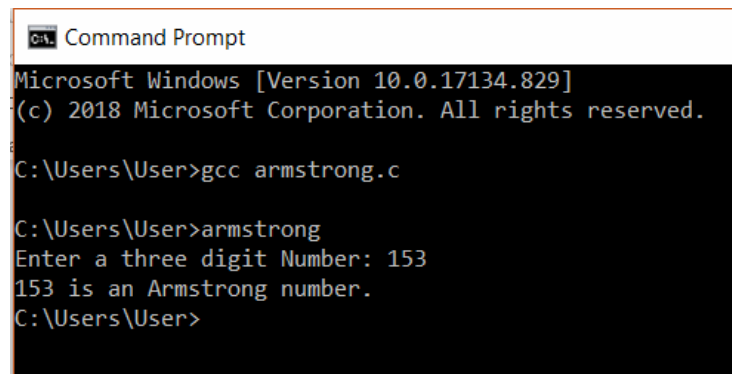
C:\Users\User>gcc armstrong.c

C:\Users\User>
```

After this, an executable file will be created in the directory that your c file exists in. Eg: `Armstrong.exe`

STEP 5:

In the next step, we can run the program. This is done by simply giving the name of the executable file without any extension. On giving this we will get an output. Here, our `Armstrong` code is executed and we got output for this code.



```
C:\> Command Prompt
Microsoft Windows [Version 10.0.17134.829]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\User>gcc armstrong.c

C:\Users\User>armstrong
Enter a three digit Number: 153
153 is an Armstrong number.
C:\Users\User>
```

DATATYPES IN C

- In C programming, data types are declarations for variables. This determines the type and size of data associated with variables.

Here's a table containing commonly used types in C programming for quick access.

Data Type	Description	Size(bytes)
int	Integers are whole numbers that can have both zero, positive and negative values but no decimal values. It can take 232 distinct states from - 2147483648 to 2147483647.	at least 2, usually 4
float	Floating type variables can hold real numbers precision of 6 digits	4
double	Floating type variables can hold real numbers with precision of 14 digits	8
char	Character data type allows a variable to store only one character.	1

VARIABLES IN C

- In programming, a variable is a container (storage area) to hold data.
- To indicate the storage area, each variable should be given a unique name (identifier). Variable names are just the symbolic representation of a memory location.

Syntax:

A variable can be declared and initialized using following syntax:

datatype variable_name = value ;

Example:

int playerScore = 95;

Here, playerScore is a variable of int type. Here, the variable is assigned an integer value 95

Rules for naming a variable

- A variable name can only have letters (both uppercase and lowercase letters), digits and underscore.

- The first letter of a variable should be either a letter or an underscore.
- There is no rule on how long a variable name (identifier) can be. However, you may run into problems in some compilers if the variable name is longer than 31 characters.

Note: You should always try to give meaningful names to variables. For example: firstName is a better variable name than fn.

- C is a strongly typed language. This means that the variable type cannot be changed once it is declared.

FORMAT SPECIFIERS

- Format Specifiers are strings used in the formatted input and output.
- The format specifiers are used in C to determine the format of input and output.
- Using this concept the compiler can understand that what type of data is in a variable while taking input using the scanf() function and printing using printf() function.

Some of the commonly used Format Specifiers are given below:

Format specifier	Description
%d or %i	It is used to print the signed integer value where signed integer means that the variable can hold both positive and negative values.
%u	It is used to print the unsigned integer value where the unsigned integer means that the variable can hold only positive value.
%o	It is used to print the octal unsigned integer where octal integer value always starts with a 0 value.
%x	It is used to print the hexadecimal unsigned integer where the hexadecimal integer value always starts with a 0x value. In this, alphabetical characters are printed in small letters such as a, b, c, etc.
%X	It is used to print the hexadecimal unsigned integer, but %X prints the alphabetical characters in uppercase such as A, B, C, etc.
%f	It is used for printing the decimal floating-point values. By default, it prints the 6 values after '.'.
%e / %E	It is used for scientific notation. It is also known as Mantissa or Exponent.
%g	It is used to print the decimal floating-point values, and it uses the fixed precision, i.e., the value after the decimal in input would be exactly the same as the value in the output.
%p	It is used to print the address in a hexadecimal form.

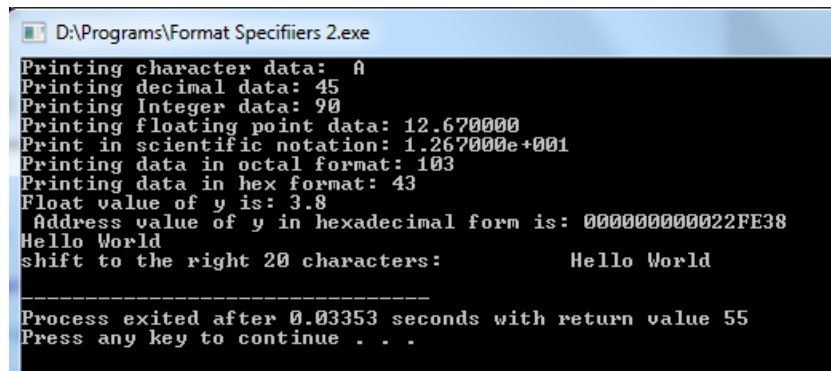
%c	It is used to print the unsigned character.
%s	It is used to print the strings.
%ld	It is used to print the long-signed integer value.

EXAMPLE:

```

1  #include <stdio.h>
2
3  main() {
4      //printing character data
5      char ch = 'A';
6      printf("Printing character data: %c\n", ch);
7
8      //print decimal or integer data with d and i
9      int x = 45, y = 90;
10     printf("Printing decimal data: %d\n", x);
11     printf("Printing Integer data: %i\n", y);
12
13     //print float value
14     float f = 12.67;
15     printf("Printing floating point data: %f\n", f);
16
17     //print in scientific notation
18     printf("Print in scientific notation: %e\n", f);
19
20     //print in octal format
21     int a = 67;
22     printf("Printing data in octal format: %o\n", a);
23
24     //print in hex format
25     printf("Printing data in hex format: %x\n", a);
26
27     float z=3.8;
28     printf("Float value of y is: %g\n ", z);
29
30     printf("Address value of y in hexadecimal form is: %p\n", &y);
31
32     char str[] = "Hello World";
33     printf("%s\n", str);
34
35     printf("shift to the right 20 characters: %20s\n", str); //
36
37 }
```

Output



```
D:\Programs\Format Specifiers 2.exe
Printing character data: A
Printing decimal data: 45
Printing Integer data: 90
Printing floating point data: 12.670000
Print in scientific notation: 1.267000e+001
Printing data in octal format: 103
Printing data in hex format: 43
Float value of y is: 3.8
Address value of y in hexadecimal form is: 000000000022FE38
Hello World
shift to the right 20 characters:      Hello World

-----
Process exited after 0.03353 seconds with return value 55
Press any key to continue . . .
```

OUTPUT IN C

- In C programming, printf() is one of the main output function. The function sends formatted output to the screen.
- The syntax of printf() function is given below:

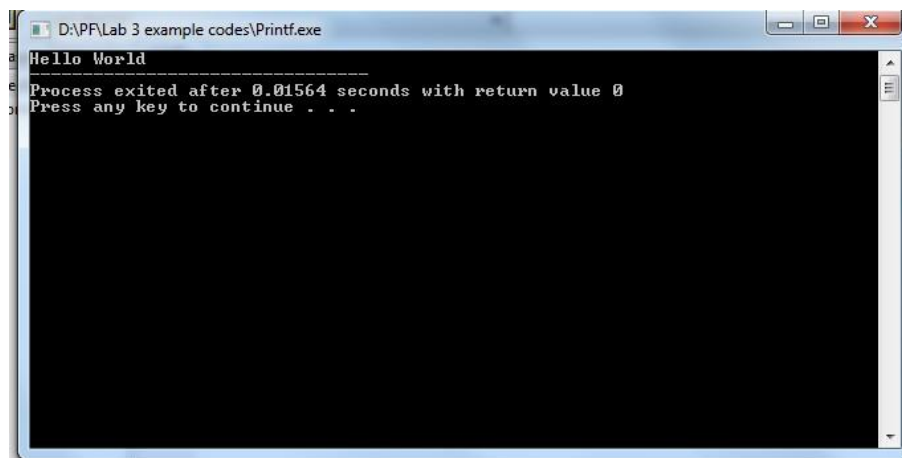
printf("format string",variable_name);

EXAMPLE

```
#include <stdio.h>

int main()
{
    printf("Hello World");
}
```

Output



```
D:\VPF\Lab 3 example codes\Printf.exe
Hello World
-----
Process exited after 0.01564 seconds with return value 0
Press any key to continue . . .
```

EXAMPLE # 2:

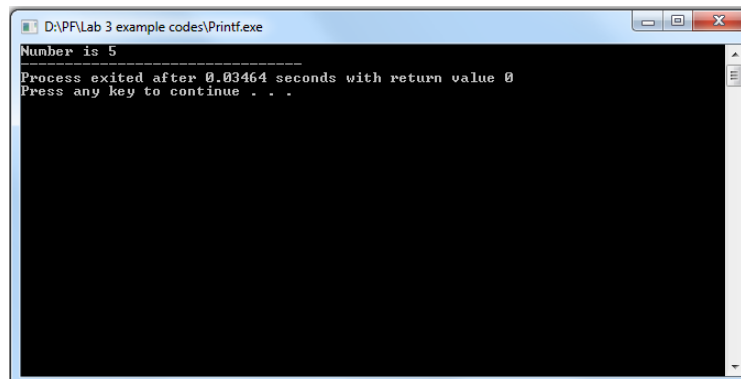
```
#include <stdio.h>

int main(){

    int testInteger=5;
    printf("Number is %d",testInteger);

}
```

Output



We use %d format specifier to print int types. Here, the %d inside the quotations will be replaced by the value of testInteger

We will study the format specifier later in this lab.

INPUT IN C

- In C programming, scanf() is one of the commonly used function to take input from the user.
- The scanf() function reads formatted input from the standard input such as keyboards.
- The syntax of printf() function is given below:

scanf("format string",variable_name);

Example:

```
testprogram.c
1  #include <stdio.h>
2  int main()
3  {
4      int testInteger;
5      printf("Enter an integer: ");
6      scanf("%d", &testInteger);
7      printf("Number = %d",testInteger);
8      return 0;
9  }
```

Output

```
Enter an integer: 2
Number = 2
-----
Process exited after 9.611 seconds with return value 0
Press any key to continue . . .
```

EXAMPLE # 2: GETTING MULTIPLE INPUTS

```
#include <stdio.h>
int main()
{
    int a;
    float b;

    printf("Enter integer and then a float: ");

    // Taking multiple inputs
    scanf("%d%f", &a, &b);

    printf("You entered %d and %f", a, b);
    return 0;
}
```

Output

```
Enter integer and then a float: 4
5.5
You entered 4 and 5.500000
-----
Process exited after 8.889 seconds with return value 0
Press any key to continue . . .
```

ESCAPE SEQUENCE

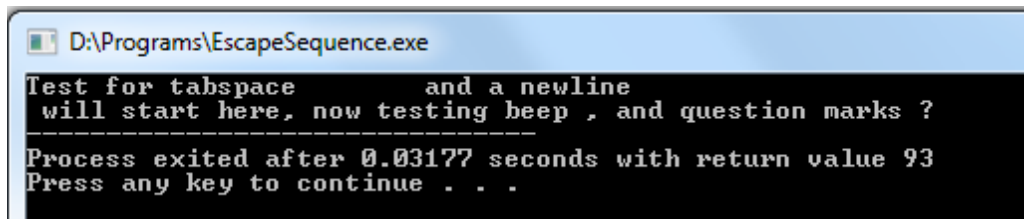
- Escape Sequences are non-printing characters.
- When a character is preceded by a backslash (\), it is called an escape sequence and it has a special meaning to the compiler. For example, \n in the following statement is a valid character and it is called a new line character.

Escape Sequence & Description
<code>\t</code> Inserts a tab in the text.
<code>\b</code> Inserts a backspace in the text.
<code>\n</code> Inserts a newline in the text.
<code>\r</code> Inserts a carriage return in the text.
<code>\f</code> Inserts a form feed in the text.
<code>\'</code> Inserts a single quote character in the text.
<code>\"</code> Inserts a double quote character in the text.
<code>\\</code> Inserts a backslash character in the text.
<code>\?</code> Inserts a question mark in the text.
<code>\a</code> Play beep or Alarm

EXAMPLE

```
1 #include <stdio.h>
2 int main() {
3     char ch1='\t';
4     char ch2 = '\n';
5     printf("Test for tabspace %c and a newline %c will start here, now testing beep \a, and question marks \?", ch1, ch2);
6     |
7 }
```

Output:



```
D:\Programs\EscapeSequence.exe
Test for tabspace      and a newline
will start here, now testing beep , and question marks ?
-----
Process exited after 0.03177 seconds with return value 93
Press any key to continue . . .
```

PRECISION SETTING IN C

Precision is specified by the number of digits after the decimal point for the outputs for float as well as double numbers.

If precision is not specified, it would be according to the default setting in the computer which is generally 6 digits.

The precision may be specified in the format specifiers place by a period(.) followed by a positive number equal to the number of digits desired.

EXAMPLE:

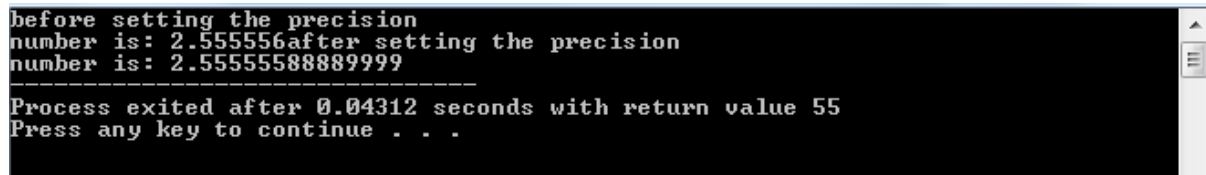
```
#include<stdio.h>

int main(){
    double a=2.55555588889999;

    printf("before setting the precision\nnumber is: %lf",a);

    printf("after setting the precision\nnumber is: %.14lf",a);
}
```

Output:

A screenshot of a terminal window with a black background and white text. The text shows the output of a C program. It first displays 'before setting the precision' followed by 'number is: 2.555556'. Then it displays 'after setting the precision' followed by 'number is: 2.55555588889999'. A dashed line separates this from the final output: 'Process exited after 0.04312 seconds with return value 55' and 'Press any key to continue . . .'.

```
before setting the precision
number is: 2.555556after setting the precision
number is: 2.55555588889999
-----
Process exited after 0.04312 seconds with return value 55
Press any key to continue . . .
```

The above will display 14 digits after the decimal point after setting the precision otherwise it will display only 6 characters even though double datatype precision is 14 digit as seen in datatypes section.